



A Case-Study Report on

Adaptive Rate Limiting System Using Queue and Hash

DATA STRUCTURES : A9503

Submitted by

N.Yokshitha sai (25881A0564)

M.Pranitha Sai Sri (25881A0535)

M.Siri Chandana (25881A0555)

Course Facilitator

Dr. V.Lokeshwari Vinya

Associate Professor

Department of Computer Science and Engineering

Vardhaman College of Engineering, Hyderabad

May 2026

CONTENTS

Case Study 1: Computer Functional Units

1	Introduction.....	
2		
1.1	Importance and Relevance to Engineering and Society	
	2	
1.2	Objectives of the Case Study.....	
	3	
2	Case Study Background.....	3
2.1	Description of the Case Scenario.....	3
2.2	Key Facts, Figures, and Stakeholders.....	
4		
2.3	Background Information	4
3	Problem Identification and Analysis.....	5
3.1	Main Concept: Computer Functional Units.....	5
3.2	Engineering Principles Applied.....	6
3.3	Analysis of the Identified Problem.....	7
4	Application of Program Outcomes.....	9
5	Sustainable Development Goals (SDGs) Mapping.....	
	 9	
5.1	SDG 9: Industry, Innovation, and Infrastructure.....	
9		
6	Proposed Solution and Recommendations.....	13
6.1	Proposed Solution.....	13
6.2		
	Recommendations.....	13
7	Conclusion and Lessons Learned	15
7.1	Conclusion.....	
15		
7.2	Lessons Learned	15

Case Study 1: Computer Functional Units

1. Introduction

1.1 Importance and Relevance to Engineering and Society

With the rapid growth of internet-based services such as web applications, mobile apps, and cloud platforms, managing user requests efficiently has become critical. Systems often face high traffic loads, which can lead to server crashes, slow performance, or security threats like Distributed Denial of Service (DDoS) attacks.

An Adaptive Rate Limiting System plays a vital role in controlling the number of requests a user can make within a specific time frame. By using data structures such as queues and hash tables, the system can efficiently track and manage requests in real time. This ensures fair usage of resources, improves system performance, and enhances security.

Such systems are widely used in APIs, banking systems, e-commerce platforms, and social media applications, making them highly relevant in modern engineering and digital society.

1.2 Objectives of the Case Study

The objectives of this case study are:

- To understand the concept of rate limiting in computer systems
- To analyze how queues and hash tables are used to implement rate limiting
- To study how adaptive mechanisms adjust limits based on system conditions
- To relate data structure concepts to real-world applications
- To highlight the importance of efficient request handling in modern systems

2. Case Study Background

2.1 Description of the Case Scenario

This case study focuses on the implementation of an Adaptive Rate Limiting System in a web server environment. Consider a popular online service that receives thousands of user requests per second. To prevent system overload and ensure fair access, the server uses a rate limiting mechanism.

Each user's requests are tracked using a hash table, where the user ID is the key. A queue is used to store timestamps of recent requests for each user. The system checks whether a user has exceeded the allowed number of requests within a given time window.

Additionally, the system adapts dynamically based on conditions such as server load or user behavior. For example, during peak traffic, the system reduces request limits, while during low traffic, it allows more requests.

2.2 Key Facts, Figures, and Stakeholders Involved

Key Facts:

- Rate limiting helps prevent server overload and ensures stability
- Queue data structure is used to maintain request order and timestamps
- Hash tables provide fast access to user-specific request data
- Adaptive systems dynamically adjust limits based on real-time conditions
- Commonly used in APIs, cloud services, and web applications

Stakeholders Involved:

- Software developers and system designers
- Network and backend engineers
- Organizations providing online services (e.g., e-commerce, banking)
- End users accessing the services

2.3 Background Information

Rate limiting is a technique used in computer systems to control the frequency of incoming requests. Traditional rate limiting methods use fixed limits, but adaptive systems improve efficiency by adjusting limits dynamically.

In this system, data structures play a crucial role:

- Queue is used to store request timestamps in chronological order, helping to remove outdated requests efficiently.
- Hash Table is used to map each user to their respective request queue, allowing quick access and updates.

This combination ensures efficient time and space complexity, making the system scalable and suitable for real-time applications. The use of data structures enables faster processing, better resource management, and improved system reliability.

3. Problem Identification and Analysis

3.1 Main Challenge: Coordination of Functional Units

The primary challenge in an adaptive rate limiting system is managing a large number of user requests efficiently in real time. Without proper control, excessive requests can overload the server, leading to performance degradation, system crashes, or security vulnerabilities such as DDoS attacks.

Another major challenge is maintaining fairness among users while dynamically adjusting limits based on varying traffic conditions. Efficient handling of multiple users simultaneously requires optimized data structures and fast access mechanisms.

3.2 Engineering Principles and Theories Applied

The following engineering principles are applied:

- **Data Structure Optimization:** Efficient use of queue and hash table for fast data access and management
- **Time Window Concept:** Tracking requests within a specific time interval
- **FIFO Principle (Queue):** Ensures proper order of request handling
- **Hashing Technique:** Enables constant-time access to user-specific data
- **Dynamic Adaptation:** Adjusting request limits based on system load and user behavior

3.3 Analysis of the Identified Problem

In real-world systems such as web servers and APIs, thousands of users send requests simultaneously. By using a hash table, each user is mapped to their own queue of request timestamps. The queue helps in maintaining recent requests and removing outdated ones efficiently.

This approach ensures that the system can quickly determine whether a user has exceeded the allowed limit. Adaptive mechanisms further enhance performance by modifying limits during peak and low traffic conditions.

Without such optimization, the system may experience delays, unfair resource allocation, and reduced reliability, highlighting the importance of efficient data structure-based design.

4. Application of Program Outcomes

This case study contributes to the following Program Outcomes (POs):

- **PO1:** Application of data structure concepts in solving real-world problems
- **PO2:** Problem analysis through efficient request handling and system optimization
- **PSO1:** Ability to design and implement scalable systems using queue and hash structures

5. Sustainable Development Goals (SDGs) Related to the Case Study

5.1 SDG 9: Industry, Innovation, and Infrastructure

Adaptive rate limiting systems contribute to reliable and efficient digital infrastructure. By preventing server overload and ensuring smooth operation of online services, they support innovation in web technologies, cloud computing, and communication systems.

Efficient resource utilization also reduces energy consumption in data centers, promoting sustainable and scalable technological growth.

6. Proposed Solution and Recommendations

6.1 Proposed Solution

- Implement rate limiting using queue to track request timestamps
- Use hash table to store and manage user-specific data efficiently
- Apply adaptive logic to dynamically adjust request limits
- Remove outdated requests to maintain an accurate time window
- Ensure real-time processing for high performance

6.2 Recommendations

- Optimize data structures for better time and space efficiency
- Use adaptive algorithms to handle varying traffic conditions
- Implement monitoring mechanisms to detect unusual activity
- Design scalable systems to support a large number of users
- Continuously test and improve system performance

7. Conclusion and Lessons Learned

7.1 Conclusion

This case study demonstrates how an adaptive rate limiting system can efficiently manage user requests using data structures like queue and hash table. By combining theoretical concepts with practical implementation, the system ensures high performance, scalability, and reliability in real-world applications such as web services and APIs.

7.2 Lessons Learned

- Efficient data structures are essential for real-time systems
- Queue and hash table provide fast and scalable solutions
- Adaptive systems improve performance under varying conditions
- Proper request management enhances system reliability and security
- Practical applications strengthen understanding of data structures

Date: _____

Name of the Course Lead: _____
(Course Lead)